

Role of Nginx in the System

Reverse proxying, static hosting, and the container-networking lessons from wiring the Angular frontend to the Chat Bridge and MCP Gateway

Prepared for systems engineers & architects · mcp-frontend / chat-bridge / mcp-gateway stack · 2026-08-14

What this document covers. A walkthrough of the work done to move the Angular frontend into the same Docker Compose deployment as the Chat Bridge and MCP Gateway, why a standalone nginx container was chosen for the frontend, what nginx is actually doing in this system, and a technical breakdown of exactly how the reverse proxy handles a request — including three real bugs encountered along the way and what they teach about proxy configuration in general.

1	System overview & what changed	3
2	The role of nginx in this system	4
3	Separate nginx vs. single-origin: the tradeoff	5
4	What the proxy actually does, step by step	7
5	Three bugs, three lessons	9
6	The compose merge, before & after	11
7	Takeaways for future services	12

1 · System overview & what changed

The stack has three runtime services. The Angular app is a pure static single-page application — it has no server-side rendering and no built-in server of its own. Something has to serve its compiled JavaScript/CSS/HTML bundle to the browser, and something has to answer its API calls. In this system those two jobs are split across two different processes:

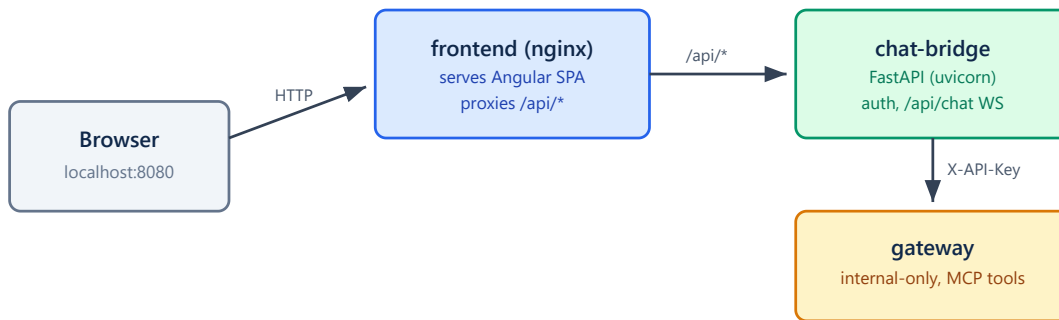


Figure 1 — The three services and their responsibilities. The browser only ever talks to nginx on port 8080; nginx and Chat Bridge are the only two hops the browser's traffic takes.

The work across the last several requests was about making this diagram *actually true* at the infrastructure level — not just on paper. Docker Compose, DNS, and nginx configuration all had to agree with it. Here is the condensed timeline:

1 Built the frontend's Dockerfile and nginx-based image

Multi-stage build: Node compiles the Angular app, nginx serves the static output. A separate `docker-compose.yml` and `nginx.conf` were added alongside it.

! Build hung, then failed: Google Fonts inlining

Angular's production optimizer tries to download Google Fonts at build time. The build sandbox has no route out to the internet, so it retried for minutes before erroring. Fixed by disabling font inlining in `angular.json`.

! Container crash-looped: nginx couldn't resolve "backend"

nginx resolves proxy hostnames at config-load time by default; no `backend` host existed, so nginx refused to even start. Fixed with Docker's embedded DNS resolver and request-time resolution (detailed in §5).

! Real deployment: signup/login failing with 502s

The running frontend container was on its own isolated Compose network (`frontend_default`), completely separate from `chat-bridge` and `gateway` 's network (`mcp-gateway_default`). No shared network, no DNS visibility — and the proxy target was pointed at a hostname (`backend`) that didn't correspond to any real service anyway.

2 Identified the real backend and its routes

The frontend repo's own dormant Express skeleton was a red herring. The real backend is `chat-bridge`, a FastAPI service on port 8001 owning `/api/auth/login`, `/api/auth/signup`, and the chat WebSocket — confirmed via its OpenAPI schema.

!

Fixed the proxy target, then found a path-mangling bug

Once pointed at the right host, requests still failed with 404/405. Chat Bridge's own logs showed every request arriving as literally `/api/`, regardless of the real path — a documented nginx gotcha involving variables in `proxy_pass` (detailed in §5).

3

Merged the frontend into the shared Compose file

`backend/mcp-gateway/docker-compose.yml` now defines all three services, with the frontend's `build.context` pointing back at the frontend repo — no files were moved. All three containers now share one Compose network with working service-name DNS, and the standalone frontend compose file was removed to avoid a conflicting duplicate deployment.

2 · The role of nginx in this system

nginx is doing two distinct jobs in the frontend container, and it's worth naming them separately because they have different failure modes and different reasons for existing.

Job 1 — Static file server for the Angular bundle

`ng build` produces plain files: HTML, JS, CSS, images. There is no server-side logic in them at all. nginx's `root` and `location /` directives serve those files directly off disk, which is close to the fastest possible way to answer a static-asset request — no application runtime, no interpreter, just the kernel's `sendfile()` path. This is nginx's original, textbook use case.

Job 2 — SPA route fallback

Angular's router handles paths like `/sign-in` or `/chat` entirely in the browser; there is no file on disk at those paths. If a user refreshes the page while on `/sign-in`, the browser asks nginx for `/sign-in` directly, and there is no such file. The `try_files $uri $uri/ /index.html;` directive is what makes this work: it tries the exact path, then the path as a directory, and if neither exists, falls back to serving `index.html` so Angular's router can boot up and interpret the URL itself.

Job 3 — Reverse proxy for `/api/*`

The Angular app's own coding convention (see the frontend repo's `src/CLAUDE.md`) is that **all** backend calls are made to same-origin paths starting with `/api`. The frontend never needs to know Chat Bridge's real hostname or port — it just calls `/api/auth/login` against whatever origin served the page. nginx is what turns that same-origin illusion into reality: it receives the request on port 8080 and forwards it, over the Docker network, to `chat-bridge:8001`. This is the classic reverse-proxy role, and it is the piece this document spends the most time on, because it is also where every bug in the timeline above actually lived.

Why same-origin matters here specifically. Chat Bridge issues its JWT as an `httpOnly` cookie (see `chat-bridge/ARCHITECTURE.md`). Cookies are scoped by origin. If the browser talked to Chat Bridge on a different origin (a different host or port than the page itself), the cookie's `SameSite` and domain rules would need to be relaxed, and the browser's automatic "attach the cookie to the WebSocket upgrade request" behavior — which the whole auth design leans on — would need CORS configuration instead. Proxying through nginx on the same origin as the page sidesteps all of that by construction.

3 · Separate nginx vs. single-origin: the tradeoff

There were, in fact, two valid ways to satisfy "the browser only ever talks to one origin." It's worth walking through both, because the choice has real operational consequences and this system's own documentation had already picked one of them before this work started.

Option A — Chat Bridge serves the frontend directly (single-origin / BFF)

Chat Bridge's own Dockerfile already supports this: build the Angular app, copy its output into `chat-bridge/frontend-dist/` before building the Chat Bridge image, and its FastAPI app mounts that directory with SPA fallback, registered *after* every `/api/*` route so those always match first. One container is then the entire public surface.

Option B — A dedicated nginx container in front, proxying to Chat Bridge (the option in use)

This is what the frontend's own Dockerfile/Compose service does: nginx serves the Angular build and reverse-proxies `/api/*` to Chat Bridge over the Docker network.

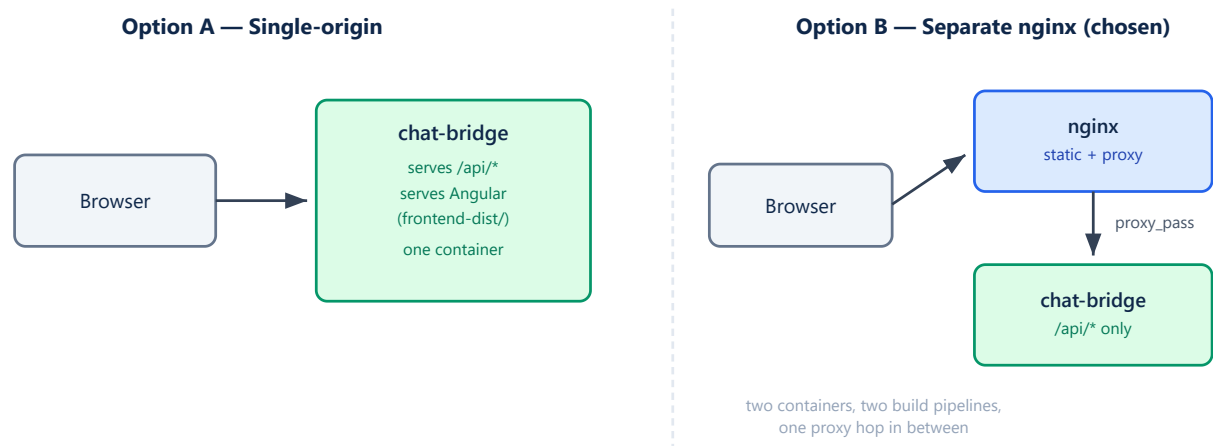


Figure 2 — The same "one public origin" requirement, satisfied two different ways.

Concern	Option A — Single-origin	Option B — Separate nginx
Moving parts	One container, one process	Two containers, a proxy hop, and proxy config as a new surface
Deploy independence	Frontend and backend release together; every frontend change touches the Chat Bridge image build	Frontend can be rebuilt/redeployed without touching Chat Bridge at all
Toolchain separation	Chat Bridge's build needs the Angular/Node toolchain too, at least at image-build time	Node stays in the frontend's own build stage; the Python image never needs it

Static asset serving	Left to Starlette's static file handling — adequate, but not nginx's purpose-built <code>sendfile()</code> path, gzip, or cache-control tuning	nginx does what it's best at: compression, cache headers, conditional requests, byte-range support
Already tested end-to-end	Yes — this is the documented v3 plan in <code>chat-bridge/ARCHITECTURE.md</code> , validated against a live Gateway	No — this path had to be built and debugged from scratch (see §5)
Failure surface	Fewer places for "it works locally, breaks in prod" — no DNS, no proxy path rewriting, no cross-container network dependency for the golden path	Three real bugs were found in this proxy path alone during setup: a DNS crash-loop, a wrong upstream hostname, and a path-truncation bug
Room to grow	Harder to bolt on a CDN, WAF, or rate-limiter later without re-architecting	nginx is a natural place to add TLS termination, rate limiting, or a CDN origin later without touching application code

This system chose Option B deliberately. When this tradeoff was raised directly, the decision was to keep the separate nginx service specifically to preserve independent frontend/backend deploys, even though it meant taking on the proxy-configuration risk and diverging from the already-tested single-origin plan. That's a legitimate call — it just means the proxy path now needs the same level of testing rigor that the single-origin path already received, since it's no longer covered by the existing "10 steps done" validation in `ARCHITECTURE.md`.

4 · What the proxy actually does, step by step

It's worth being precise about this, because "nginx proxies /api to the backend" hides several discrete steps, each of which was a place a bug actually surfaced during setup.

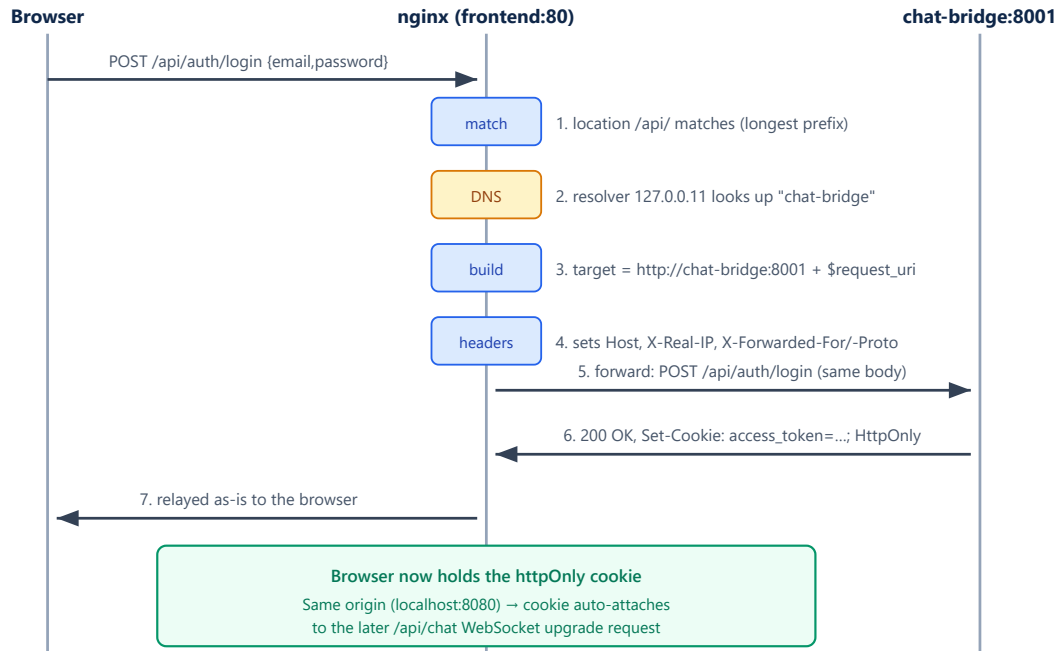


Figure 3 — The full lifecycle of a proxied login request. Steps 2–4 are where all three bugs in §5 actually happened.

The nginx.conf, annotated

```
location /api/ {
    # Do DNS lookups at request time, not at nginx startup.
    resolver 127.0.0.11 valid=10s;

    # Putting the host in a variable is what forces nginx to defer
    # resolution -- see the "static vs. variable proxy_pass" box below.
    set $backend_upstream http://chat-bridge:8001;

    # $request_uri is the client's ORIGINAL path + query string,
    # untouched by location matching. Appending it here is what
    # makes "/api/auth/login" actually arrive as "/api/auth/login".
    proxy_pass $backend_upstream$request_uri;

    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Header set	Why it matters
Host	Forwards the original <code>Host: localhost:8080</code> instead of nginx substituting <code>chat-bridge:8001</code> , so any host-based logic on the backend (or in logs) reflects what the browser actually requested.
X-Real-IP	The backend sees every request as coming from nginx's own container IP unless told otherwise; this preserves the real client address for logging/rate-limiting.
X-Forwarded-For	The standard chain-of-proxies header — useful the moment there's more than one hop (a load balancer or CDN in front of nginx, for example).
X-Forwarded-Proto	Tells the backend whether the original request was HTTP or HTTPS, since that information is otherwise lost once nginx terminates the connection.

5 · Three bugs, three lessons

Each of these was found empirically during setup, not by inspection. They're worth walking through in detail because none of them are specific to this system — they're general nginx/Docker gotchas that will resurface on any similar reverse-proxy setup.

Bug 1 — Startup-time DNS resolution crash-loops the container

Symptom: `nginx: [emerg] host not found in upstream "backend"` — the container exited immediately on every start.

By default, when `proxy_pass` names a hostname directly (`proxy_pass http://backend:9000;`), nginx resolves that hostname **once, when the configuration is loaded** — effectively, at container startup. If the target host doesn't exist yet (or ever, in a misconfigured case), nginx treats it as a fatal configuration error and refuses to start at all, rather than starting and returning errors only for requests that hit that location.

Fix: put the upstream URL in a variable and add a `resolver` directive pointing at Docker's embedded DNS server (`127.0.0.11` , available inside every container on a user-defined Docker network). This forces nginx to resolve the hostname *per request* instead of once at startup, so a temporarily-unreachable or renamed backend degrades to per-request `502` s instead of taking the whole nginx process down.

Before (crash-loops)

```
location /api/ {
    proxy_pass http://backend:9000;
}
```

After (degrades gracefully)

```
resolver 127.0.0.11 valid=10s;
location /api/ {
    set $up http://chat-bridge:8001;
    proxy_pass $up$request_uri;
}
```

Bug 2 — The wrong hostname entirely

Symptom: `502 Bad Gateway` on every `/api` call, even after Bug 1 was fixed.

This wasn't a proxy-mechanics bug at all — it was a wrong assumption. The frontend repo's own documentation described an Express backend on port 9000, but that Express skeleton is deliberately unused; the real backend is a separate FastAPI service, `chat-bridge` , on port 8001. The service also wasn't reachable on the network the frontend container had joined at all — two separate Compose projects had created two separate Docker networks with no bridge between them. Both the wrong hostname and the missing network connectivity had to be fixed before any request could reach a real backend.

Lesson: a reverse proxy's target is a claim about system topology, not just a config value. Verifying it means checking what's *actually running* (`docker ps` , the target service's own OpenAPI schema, its logs) rather than trusting a name that sounds plausible.

Bug 3 — Variables in `proxy_pass` silently drop the request path

Symptom: `404 / 405` on every `/api/*` route, even `/api/auth/login` , which genuinely exists on the backend.

This is the subtlest of the three, and the one most worth remembering. Normally, when `proxy_pass` is given a URI (e.g. `proxy_pass http://host/api/;`), nginx automatically replaces the part of the request URI that matched the `location` prefix with that URI, carrying the rest of the original path along. A request for `/api/auth/login` hitting `location /api/` correctly becomes `/api/auth/login` on the upstream.

But this automatic rewriting only happens when the target is a static string. The moment any part of the `proxy_pass` target comes from a variable — exactly what Bug 1's fix introduced — nginx stops doing that rewrite. The configured URI is used *literally*, for every request under that location, regardless of what actually followed `/api/` in the original request.

Before: literal target

Client requests:
`/api/auth/login`
chat-bridge actually receives:
`/api/ (path lost)`

After: + `$request_uri`

Client requests:
`/api/auth/login`
chat-bridge actually receives:
`/api/auth/login (preserved)`

Figure 4 — Confirmed empirically from chat-bridge's own access log: every proxied request, regardless of the real path requested, arrived as `GET /api/` or `POST /api/` until `$request_uri` was added.

Fix: explicitly append `$request_uri` — the client's original, untouched path and query string — to the variable-based target:

```
set $backend_upstream http://chat-bridge:8001;  
proxy_pass $backend_upstream$request_uri; # not just $backend_upstream/api/
```

Because this location's prefix (`/api/`) already matches what chat-bridge itself expects as a prefix, using `$request_uri` outright is correct here without any further rewriting. In a setup where the proxy needed to *strip* a prefix before forwarding, the fix would need to reconstruct the remaining path manually (e.g. via a captured regex location) rather than relying on the automatic behavior, since that automatic behavior is exactly what variables disable.

General rule to carry forward: the moment a `resolver` and a variable are introduced to make DNS resolution lazy, path handling stops being automatic and has to be made explicit again with

`$request_uri` (or equivalent). These two nginx behaviors interact in a way that isn't obvious from either one in isolation — and this configuration will need both any time a proxy target's hostname might not exist yet when nginx starts, which is the normal case in container orchestration.

6 · The compose merge, before & after

Fixing the nginx configuration wasn't enough on its own — nginx also needed a network path to `chat-bridge` to resolve at all. Docker Compose creates one network per project by default, named after the directory the compose file lives in. Two separate `docker-compose.yml` files, run from two separate directories, produce two separate, mutually invisible networks — even if both are on the same Docker host.

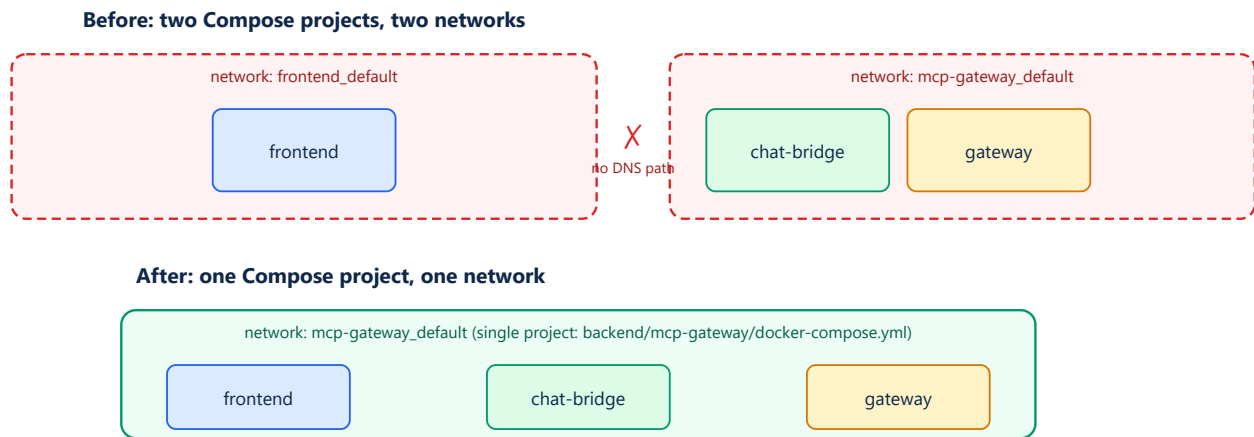


Figure 5 — Two isolated networks (interim state, patched with an external-network reference) versus one shared network under a single Compose project (final state).

How the merge was done without moving files

Docker Compose resolves a service's `build.context` relative to the compose file's own location, not relative to the current working directory. That means a service can be added to `backend/mcp-gateway/docker-compose.yml` whose build still lives entirely inside the sibling `frontend/` directory:

```
frontend:
  build:
    context: ../../frontend      # resolved relative to this compose file
    dockerfile: Dockerfile
  ports:
    - "8080:80"
  volumes:
    - ../../frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    - nginx-logs:/var/log/nginx
  depends_on:
    - chat-bridge
```

No files were relocated. The frontend repository keeps full ownership of its own `Dockerfile`, `nginx.conf`, and source tree; the compose file simply points at it from outside. This mirrors exactly how the `chat-bridge` and `gateway` services were already declared in that same file — both use `context: ../../chat-bridge` and `context: ..` respectively, for the same reason.

Once all three services are declared in one compose file, Compose puts them on one shared network by default, and each container becomes reachable from the others by its service name — `chat-bridge` resolves correctly from inside the `frontend` container with no further network configuration required. The interim fix (explicitly attaching to an `external: true` network by name) is no longer necessary once the projects are merged, and was removed along with the now-redundant standalone `frontend/docker-compose.yml` .

7 · Takeaways for future services

1. **A reverse proxy is infrastructure, and it deserves the same scrutiny as application code.**

Three distinct, real bugs lived entirely in ~15 lines of `nginx.conf`. None of them were visible from reading the config casually — each needed an empirical check (a failing build, a crash-looping container, an upstream's own access log) to surface.

2. **Lazy DNS resolution and automatic path rewriting are coupled in nginx.** Any proxy pointed at a container hostname that might not exist at nginx-startup time needs a `resolver` and a variable-based target — and the moment that's true, `$request_uri` (or equivalent manual path reconstruction) has to be added back explicitly, or paths silently collapse to the location prefix.

3. **Verify proxy targets against what's actually running, not against what documentation implies.** The frontend's own docs pointed at a plausible-sounding but unused backend. `docker ps`, the target's OpenAPI schema, and its own logs were what actually confirmed the real target.

4. **Compose network boundaries are per-project by default.** Services that need to talk to each other via service-name DNS should live in the same compose file (or explicitly share an `external: true` network) — and a single compose file is the more durable choice, since it doesn't depend on another team's project directory name or on that other project staying up.

5. **"Separate nginx vs. single-origin" is a real architectural decision, not just a config detail.**

This system already had a tested single-origin path documented and built. Choosing the separate-nginx path instead, to preserve independent frontend/backend deploys, was a legitimate tradeoff — but it forfeits that existing test coverage and takes on the proxy layer as new, first-class surface area that now needs its own equivalent testing and monitoring going forward.